

Da tabela ao **container**

Um guia sem pressa sobre bancos relacionais e não relacionais, o que o Docker realmente faz na sua máquina, e onde essa combinação toda aparece fora da sala de aula — escrito a partir dos dois exercícios que você rodou com MySQL e MongoDB.

01

O que é um banco de dados, sem enrolação

Todo programa que "lembra" de alguma coisa entre um uso e outro — seu usuário, seu carrinho de compras, sua conversa — precisa guardar isso em algum lugar permanente. Um banco de dados é só um software especializado nisso: guardar informação de forma organizada, permitir buscar rápido, e não perder nada quando o programa fecha.

Você já faz isso manualmente numa planilha do Excel. A diferença é que um banco de dados foi construído para ser acessado por um programa (não por uma pessoa clicando), lidar com milhares de operações por segundo, e garantir que os dados não se corrompam mesmo com várias pessoas mexendo ao mesmo tempo.

Existem dezenas de bancos de dados diferentes, mas quase todos se encaixam em duas grandes famílias — e foi exatamente uma de cada que você usou nos dois exercícios.

O terminal, por dentro

O terminal é só outro jeito de dar instrução pro computador: em vez de clicar em ícones, você digita o que quer que aconteça e aperta Enter. É mais rápido e mais poderoso pra programação — quase todo tutorial técnico, curso e ferramenta de desenvolvedor passa por ele em algum momento — mas o "idioma" dele não é óbvio de início.

Lendo o prompt

Repare na linha que aparece toda vez que o terminal está esperando um comando seu:

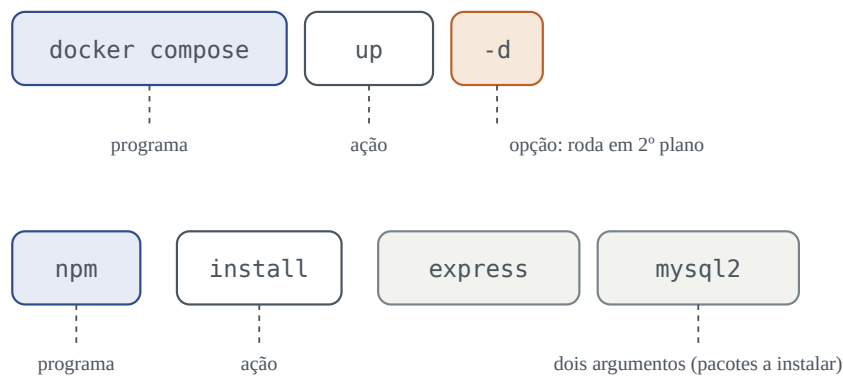
```
shibuya@MacBook-Air-de-Gabriella mysql-app %
```

PARTE	O QUE É
shibuya	seu usuário no Mac
MacBook-Air-de-Gabriella	o nome do computador
mysql-app	a pasta onde você está "parada" agora — se você criar um arquivo sem indicar caminho, ele nasce aqui
%	sinal de que o terminal está livre e esperando um comando (em outros sistemas costuma aparecer \$)

Esse % é a assinatura do **zsh**, o programa que interpreta tudo que você digita — é ele quem lê **cd**, entende que é "mudar de pasta", e executa. Esse tipo de programa se chama **shell**. O "Terminal" é só a janela; o zsh é quem realmente trabalha por dentro dela.

A anatomia de um comando

Praticamente todo comando segue o mesmo formato: **programa** → **ação** (opcional) → **opções** (começam com -, mudam o comportamento) → **argumento** (o "alvo" do comando). Dois comandos que você já rodou hoje, quebrados em pedaços:



O que a imagem mostra: os mesmos quatro papéis se repetem em quase todo comando — o programa que você está chamando, a ação que ele deve tomar, opções que ajustam o comportamento (sempre com `-` na frente) e os argumentos, que são o "alvo" da ação.

Se movendo entre pastas

Antes do Docker e do Node entrarem em cena, o terminal já precisa saber navegar entre pastas — é basicamente o Finder, só que digitado:

COMANDO	O QUE FAZ
<code>pwd</code>	Mostra em qual pasta você está agora (print working directory)
<code>ls</code>	Lista o que existe dentro da pasta atual
<code>ls -la</code>	Mesma coisa, mas mostrando também arquivos ocultos e detalhes (dono, tamanho, data)
<code>cd nome-da-pasta</code>	Entra em uma pasta (Change Directory)
<code>cd ..</code>	Volta uma pasta pra trás
<code>cd ~</code>	Vai direto pra sua pasta pessoal (Home)
<code>mkdir nome</code>	Cria uma pasta nova
<code>clear</code>	Limpa a tela (não apaga nada, só "arruma a bagunça" visual)
<code>↑</code> (seta pra cima)	Repete o último comando digitado — poupa muita digitação
<code>Tab</code>	Autocompleta o nome do arquivo/pasta que você começou a digitar

Foi exatamente essa lógica que usamos pra chegar nas pastas dos exercícios: `cd ~/Documents/aula-bd/mysql-app` significa "vá da minha pasta pessoal (~), entre em Documents, depois em aula-bd, depois em mysql-app" — tudo em um comando só, separado por barras.

Terminal, parte 2: o resto da caixa de ferramentas

A parte anterior cobre o suficiente pra sobreviver nos exercícios. Esses aqui são os comandos que aparecem assim que você começa a fazer mais coisa sozinha — sem precisar decorar tudo de uma vez, só pra saber que existem quando precisar.

Mexendo em arquivos direto pelo terminal

COMANDO	O QUE FAZ
<code>cat arquivo.txt</code>	Joga o conteúdo inteiro do arquivo na tela
<code>less arquivo.txt</code>	Mostra o conteúdo página por página — use as setas pra rolar e q pra sair
<code>touch arquivo.js</code>	Cria um arquivo vazio com esse nome
<code>cp origem destino</code>	Copia um arquivo
<code>mv origem destino</code>	Move OU renomeia um arquivo (é o mesmo comando pras duas coisas)
<code>rm arquivo</code>	Apaga um arquivo — sem lixeira , não tem como desfazer

O `rm` merece respeito: diferente de arrastar pra Lixeira no Finder, apagar pelo terminal é definitivo. Antes de rodar `rm` em qualquer coisa, vale rodar `pwd` e `ls` primeiro pra ter certeza absoluta de onde você está e o que vai apagar.

Redirecionamento e pipes: fazendo comandos conversarem

Dois símbolos abrem um mundo de possibilidades: `>` pega a saída de um comando e salva num arquivo (em vez de mostrar na tela), e `|` (chamado de "pipe") pega a saída de um comando e usa como entrada do próximo.



O que a imagem mostra: `cat index.js | grep console > resultado.txt`
— mostra o arquivo, filtra só as linhas que têm "console" e salva o resultado num arquivo novo, tudo numa linha só, sem precisar de três passos separados.

O PATH: por que o npm "sumiu" até você abrir um terminal novo

Isso não foi coincidência — foi uma peça que faltava no quebra-cabeça. Quando você digita `npm`, o terminal não sabe magicamente onde esse programa está guardado no seu Mac: ele procura numa lista de pastas chamada **PATH** (uma variável de ambiente). Quando você instalou o Node.js, o instalador adicionou a pasta dele nessa lista — mas o Terminal que já estava aberto tinha carregado a lista *antes* disso acontecer, então ainda não sabia. Um terminal novo lê a lista atualizada, e aí encontra o `npm`.

Pra ver essa lista você mesma: `echo $PATH`. O `$` na frente de um nome sempre significa "o valor guardado nessa variável", não o texto literal.

Buscando coisas

COMANDO	O QUE FAZ
<code>grep "termo" arquivo</code>	Procura um texto dentro de um arquivo e mostra as linhas onde ele aparece
<code>find . -name "*.js"</code>	Procura, a partir da pasta atual (<code>.</code>), todos os arquivos que terminam em <code>.js</code>
<code>which docker</code>	Mostra em qual pasta um programa está instalado (ou confirma que ele existe no PATH)

Histórico, atalhos e permissões

COMANDO / ATALHO	O QUE FAZ
<code>history</code>	Lista os últimos comandos que você digitou nessa sessão
<code>Ctrl + R</code>	Busca reversa: começa a digitar parte de um comando antigo e ele aparece
<code>Ctrl + C</code>	Interrompe o que está rodando naquele terminal (já usamos bastante)
<code>Ctrl + L</code>	Limpa a tela — mesmo efeito do comando <code>clear</code>
<code>sudo comando</code>	Roda o comando "como administrador" — o Mac pede sua senha; use só quando um erro pedir explicitamente
<code>chmod +x arquivo</code>	Torna um arquivo executável — comum antes de rodar scripts baixados da internet

Sobre o sudo: ele te dá poder de sobrescrever praticamente qualquer proteção do sistema. É seguro quando um tutorial confiável pede explicitamente; é arriscado copiar e colar um comando com `sudo` de um lugar que você não confia totalmente — nesse caso, ele roda com muito mais permissão pra causar estrago.

Relacional × não relacional

SQL · RELACIONAL

Um banco relacional (o MySQL do primeiro exercício) organiza tudo em **tabelas** — linhas e colunas, exatamente como uma planilha. Cada tabela tem um formato fixo definido antes de você inserir qualquer dado, e tabelas diferentes se conectam entre si por **chaves**: um pedido "aponta" para o cliente que fez ele, por exemplo.

id	numero_decimal	numero_binario
1	55	110111
2	7	111
3	82	1010010

† a própria tabela `conversoes` que você criou no phpMyAdmin.

Onde o modelo relacional é a escolha certa

Sistema bancário

Saldo e transferência exigem que o dinheiro nunca "suma" no meio do caminho — é o tipo de garantia que bancos relacionais foram feitos para dar.

Loja on-line (pedidos)

Um pedido se liga a um cliente, a vários produtos e a um endereço de entrega — relações claras entre entidades.

Matrícula acadêmica

Aluno, disciplina e turma são entidades separadas que se cruzam de formas bem definidas.

ERP de empresa

Financeiro, estoque e RH precisam bater entre si — consistência importa mais do que velocidade bruta.

NOSQL · NÃO RELACIONAL

Um banco não relacional (o MongoDB do segundo exercício) abandona a ideia de tabela fixa. O "NoSQL" não é uma linguagem — é um guarda-chuva para vários modelos diferentes, cada um resolvendo um problema que o modelo relacional resolve mal ou devagar.


```
{ "_id": ObjectId("64f..."), "numero_decimal": 10, "numero_binario": "1010" }
```

↑ um documento da coleção **conversoes** no seu banco Mongo — sem colunas fixas, cada documento pode ter um formato diferente.

DOCUMENTO

MongoDB

Guarda registros como JSON. Ótimo quando cada item tem atributos diferentes dos outros.

EXEMPLO REAL

Catálogo de produtos de e-commerce — uma camiseta tem cor e tamanho, um livro tem autor e ISBN, um só banco guarda os dois sem forçar as mesmas colunas.

CHAVE-VALOR

Redis

Cada dado é um par **chave** → **valor**, lido e escrito quase instantaneamente.

EXEMPLO REAL

Guardar a sessão de login de cada usuário, ou o cache de uma página muito acessada, pra não bater no banco principal toda hora.

GRAFO

Neo4j

Modela dados como nós conectados por relações — o que importa são as conexões, não as tabelas.

EXEMPLO REAL

"Pessoas que você talvez conheça" numa rede social, ou detecção de fraude olhando conexões suspeitas entre contas.

COLUNAR

Cassandra

Armazena por coluna em vez de por linha, o que acelera muito consultas sobre volumes enormes de dados.

EXEMPLO REAL

A Netflix usa esse modelo para analisar bilhões de eventos de log e encontrar falhas rapidamente.

Nenhum dos dois é "melhor" de forma absoluta — **relacional** ganha quando integridade e relações claras importam mais; **não relacional** ganha quando o formato dos dados varia muito ou a velocidade de leitura/escrita é crítica. Sistemas grandes de verdade costumam usar os dois ao mesmo tempo, cada um no que faz melhor.

Os 4 tipos de comando SQL

Toda instrução SQL se encaixa em um destes quatro grupos, de acordo com o que ela faz no banco:

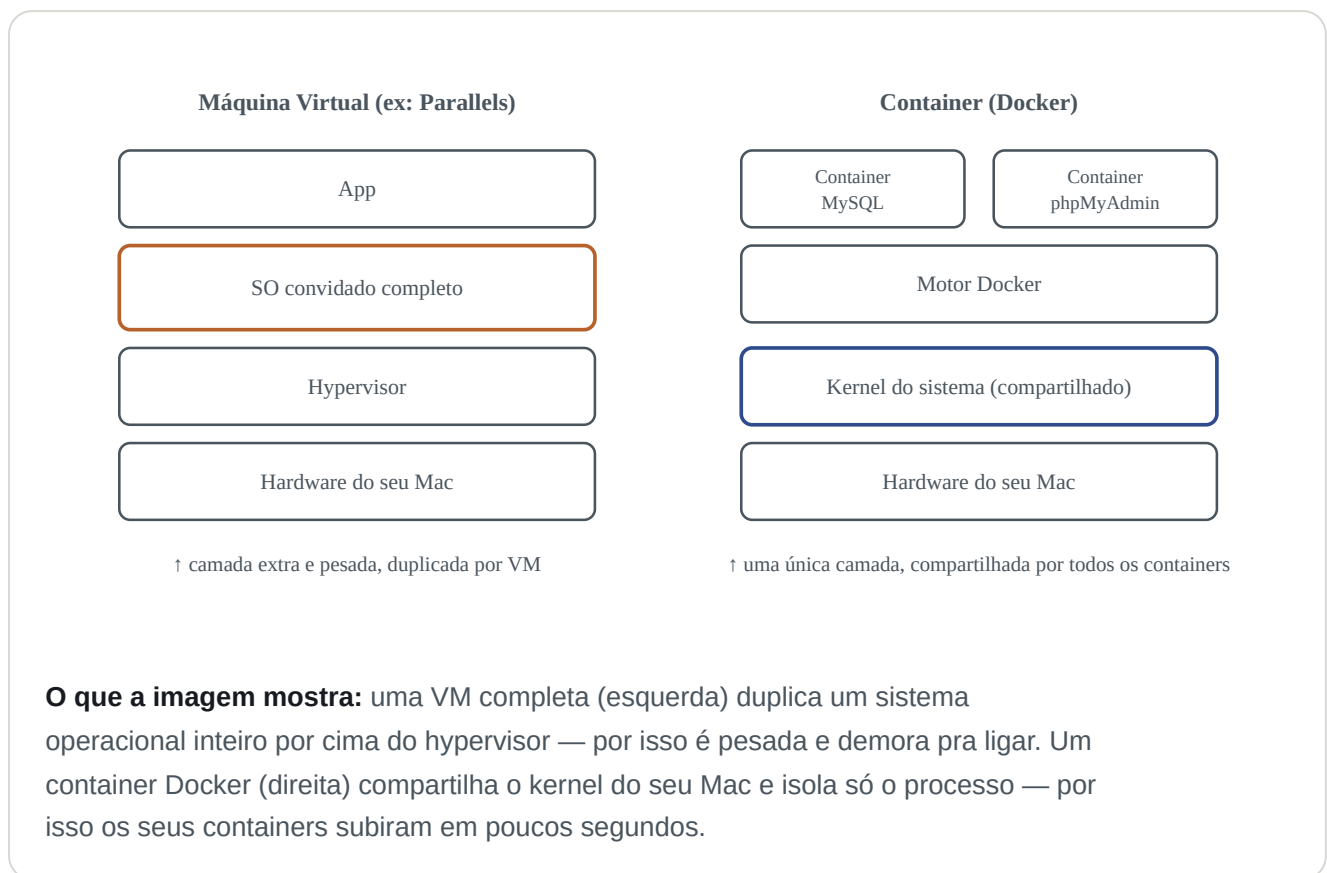
SIGLA	NOME	PARA QUE SERVE	COMANDOS
DDL	Data Definition Language	Define a <i>estrutura</i> do banco — criar, alterar e apagar tabelas	CREATE, ALTER, DROP
DML	Data Manipulation Language	Mexe nos <i>dados</i> dentro das tabelas	SELECT, INSERT, UPDATE, DELETE
DCL	Data Control Language	Controla <i>quem</i> pode acessar o quê	GRANT, REVOKE
TCL	Transaction Control Language	Garante que um grupo de operações aconteça por completo ou não aconteça	COMMIT, ROLLBACK

O `CREATE TABLE conversoes (...)` que você rodou no phpMyAdmin é

DDL — definiu a estrutura. Quando a aplicação Node faz `INSERT INTO conversoes (...)` a cada conversão, isso é **DML** — está manipulando os dados que já existem dentro dessa estrutura.

O que o Docker faz de verdade

Um **container** não é um "PC virtual" completo — é mais leve que isso. Ele empacota um programa (o MySQL, por exemplo) junto com tudo que ele precisa pra rodar, isolado do resto do seu sistema, mas *compartilhando* o mesmo kernel do seu computador em vez de simular um computador inteiro do zero.

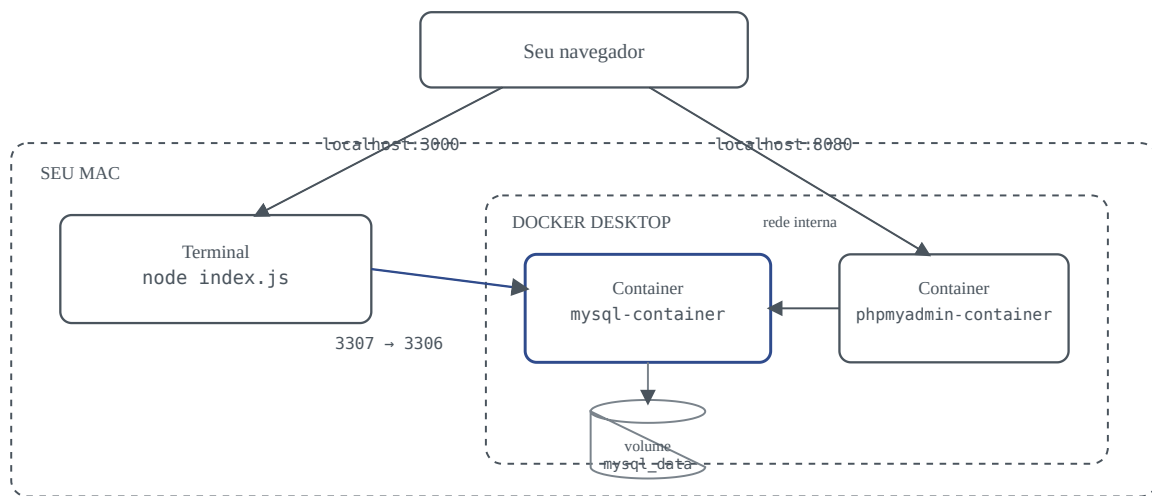


E os dados, ocupam espaço de verdade?

Sim, os dois tipos, de duas formas diferentes:

Imagens — quando você rodou `docker compose up -d`, o Docker baixou o "molde" do MySQL e do phpMyAdmin (cada um com centenas de MB) e guardou no disco do seu Mac, dentro da área que o Docker Desktop administra.

Volumes — as linhas que você inseriu de verdade (os números convertidos) ficam gravadas num *volume*, um pedaço de disco que sobrevive mesmo se você apagar e recriar o container. Só some se você apagar o volume também (`docker compose down -v`).



O que a imagem mostra: o mapa completo do exercício de MySQL. Sua aplicação Node roda direto no Mac (fora do Docker) e fala com o banco pela porta mapeada; o phpMyAdmin, dentro do Docker, fala com o banco pela rede interna dos containers; e tudo que é gravado vai parar no volume, em disco.

A receita: Node + Docker + Banco

Foi essa sequência que você seguiu duas vezes hoje — uma com MySQL, outra com MongoDB. É basicamente o mesmo roteiro pra qualquer combinação de linguagem + banco, então vale guardar como checklist.

01 Escolher o banco e escrever o `docker-compose.yml`

Relacional → imagem do MySQL/Postgres + uma ferramenta de admin (phpMyAdmin). Não relacional → imagem do Mongo/Redis + a ferramenta correspondente (Mongo Express).

02 Subir os containers

`docker compose up -d` — baixa as imagens (só na primeira vez) e liga tudo em segundo plano.

03 Criar a estrutura (só em banco relacional)

`CREATE TABLE` pela interface de admin. Bancos não relacionais criam a coleção sozinhos, na primeira inserção.

04 Escrever o `index.js` com as mesmas credenciais

Usuário, senha e nome do banco no código precisam ser *idênticos* aos definidos no `docker-compose.yml` — é esse casamento que faz a conexão funcionar.

05 Instalar as dependências

`npm init -y` seguido de `npm install` das bibliotecas que o código importa.

06 Rodar e testar

`node index.js`, depois acessar a rota no navegador e conferir os dados na ferramenta de admin.

Onde isso aparece no mundo real

Essa combinação inteira — aplicação em uma linguagem, banco de dados rodando em container, subindo tudo com um único comando — não é um exercício isolado de faculdade. É como boa parte das equipes de desenvolvimento profissional trabalha no dia a dia, antes mesmo de qualquer coisa ir para produção.

Um exemplo concreto: imagine um app de entregas. O time provavelmente teria uma API em Node ou Python cuidando de pedidos (banco **relacional**, porque pagamento e estoque não podem "quase bater"), um banco **chave-valor** tipo Redis guardando a localização em tempo real dos entregadores (precisa ser rapidíssimo, e não precisa durar pra sempre), e talvez um banco de **documentos** guardando o histórico de avaliações dos restaurantes, cada um com campos diferentes.

Todo esse conjunto seria descrito num `docker-compose.yml` parecido com o que você escreveu — e cada desenvolvedor do time sobe a mesma configuração exata na própria máquina, sem precisar instalar nada manualmente nem brigar com "na minha máquina funciona".

Ambiente de desenvolvimento local

Times inteiros usam `docker-compose up` pra replicar, no laptop de cada um, o mesmo conjunto de serviços que roda em produção.

Testes automatizados (CI/CD)

Pipelines de teste sobem um banco descartável em container, rodam os testes contra ele, e derrubam tudo no final — sem sujar nenhum banco real.

Portfólio e entrevistas técnicas

Saber demonstrar uma API conectada a um banco, rodando via Docker, é exatamente o tipo de projeto que aparece em testes técnicos de vaga.

Microsserviços

Sistemas grandes dividem funções em vários serviços pequenos, cada um com seu próprio banco — o padrão que você praticou, multiplicado.

Cola rápida de comandos

COMANDO	O QUE FAZ
<code>docker compose up -d</code>	Cria e inicia os containers do <code>docker-compose.yml</code> em segundo plano
<code>docker compose down</code>	Para e remove os containers (os dados no volume continuam salvos)
<code>docker compose down -v</code>	Igual acima, mas também apaga os volumes — cuidado, isso apaga os dados
<code>docker ps</code>	Lista os containers que estão rodando agora
<code>docker compose logs -f</code>	Mostra os logs dos containers em tempo real — ótimo pra debugar
<code>npm init -y</code>	Cria o <code>package.json</code> com as configurações padrão
<code>npm install nome-do-pacote</code>	Baixa e instala uma biblioteca dentro do projeto
<code>node index.js</code>	Executa a aplicação Node.js
<code>Ctrl + C</code>	Para a aplicação que está rodando naquele terminal

Erros comuns (você já resolveu dois deles hoje)

MENSAGEM	O QUE SIGNIFICA	COMO RESOLVER
<code>address already in use</code>	Outro programa já está usando aquela porta no seu Mac	Trocar a porta no docker-compose.yml (ex: 3307:3306)
<code>command not found: npm</code>	O Node.js ainda não está instalado nessa máquina	Instalar em nodejs.org e abrir um terminal novo
<code>Cannot find module 'express'</code>	As dependências do projeto não foram instaladas	Rodar <code>npm install</code> na pasta do projeto
<code>ECONNREFUSED</code>	O container do banco ainda não terminou de iniciar	Esperar alguns segundos após o <code>up -d</code> antes de rodar a aplicação
<code>erro 400 / 500 no navegador</code>	O valor enviado é inválido, ou o banco recusou a operação	Conferir a URL e ler o erro no terminal onde o node está rodando

Toda mensagem de erro do terminal carrega a resposta dentro dela — o primeiro passo é sempre **ler a última linha antes de reclamar do computador**. Foi assim que resolvemos os dois erros de hoje: a porta ocupada e o npm não encontrado.

Guia construído a partir dos exercícios de Introdução a Bancos de Dados (MySQL + Docker, MongoDB + Docker) — para consulta antes de provas ou novos exercícios.